TASK 2 — PROJECT REPORT

Ultrasonic Parking Sensor

Simulation using Arduino Uno & HC-SR04 in Proteus 8

Software	Proteus 8 Professional — Schematic Capture
Microcontroller	Arduino Uno R3 (ATmega328P)
Sensor	HC-SR04 Ultrasonic Distance Sensor
Components	4 LEDs (2× Green, 1× Yellow, 1× Red), Buzzer, Virtual Terminal
Communication	Serial UART @ 9600 baud
Subject	Embedded Systems / Arduino Simulation

1. Introduction

This report documents Task 2: the design, implementation, and simulation of an Ultrasonic Parking Sensor system using an Arduino Uno R3 microcontroller and an HC-SR04 ultrasonic distance sensor, simulated in Proteus 8 Professional. The system mimics a real-world car parking sensor by providing visual (LEDs) and audible (buzzer) feedback based on the distance of an object from the sensor, and prints real-time distance readings to a Virtual Terminal via serial communication.

The simulation demonstrates five distinct distance zones, each triggering a different combination of LED states and buzzer beep patterns. The Virtual Terminal is configured to print a new line only when the distance changes by more than 2 cm or when the zone changes, preventing unnecessary output spam.

2. Components Used

Component	Quantity	Label in Proteus	Purpose
Arduino Uno R3	1	ARD1	Main microcontroller
HC-SR04	1	U1	Ultrasonic distance sensor
LED-GREEN	2	D1, D2	Safe / Near zone indicator
LED-YELLOW	1	D4	Caution zone indicator
LED-RED	1	D3	Warning / Stop zone indicator
BUZZER	1	BUZ1	Audible distance alert

Virtual Terminal	1	—	Serial monitor (9600 baud)

3. Pin Connections

Arduino Pin	Connected To	Description
5V	HC-SR04 VCC	Power supply for sensor
GND	HC-SR04 GND, all LED cathodes, Buzzer –	Common ground
D2	HC-SR04 TRIG	Trigger pulse to sensor
D3	HC-SR04 ECHO	Echo return from sensor
D4	LED-GREEN (D1)	Safe zone indicator
D5	LED-GREEN (D2)	Near zone indicator
D6	LED-YELLOW (D4)	Caution zone indicator
D7	LED-RED (D3)	Warning / Stop indicator
D8	Buzzer + pin	Buzzer control
TX/D1	Virtual Terminal RXD	Serial output @ 9600 baud

4. Distance Zones & Behaviour

Zone	Distance	LEDs ON	Buzzer	Serial Output
SAFE	cm > 50	Green D1 only	Silent	SAFE [*---]
NEAR	30–50 cm	Green D1 + D2	Slow (100ms on)	NEAR [**--]
CAUTION	15–30 cm	D1 + D2 + Yellow D4	Medium (200ms on)	CAUTION [***-]
WARNING	5–15 cm	All 4 LEDs	Fast (300ms on)	WARNING! [****]
STOP	cm < 5	Red D3 flashing	Solid ON	>>> STOP! STOP! STOP! <<<

5. Arduino Source Code

The following code was compiled in Arduino IDE, exported as a .hex file, and loaded into the Arduino Uno R3 component in Proteus via the Program File property.

```
#define TRIG    2
#define ECHO    3
#define LED1    4   // green  - safe
#define LED2    5   // green  - near
#define LED3    6   // yellow - caution
#define LED4    7   // red    - stop/warning
#define BUZZER  8

long lastDistance = -1;   // stores previous reading
String lastZone   = "";   // stores previous zone

void setup() {
  Serial.begin(9600);
  pinMode(TRIG,   OUTPUT);
  pinMode(ECHO,   INPUT);
  pinMode(LED1,   OUTPUT);
  pinMode(LED2,   OUTPUT);
  pinMode(LED3,   OUTPUT);
  pinMode(LED4,   OUTPUT);
  pinMode(BUZZER, OUTPUT);
  Serial.println("=== PARKING SENSOR READY ===");
  Serial.println("Waiting for distance change...");
  Serial.println("-----------------------------");
}

void allLedsOff() {
  digitalWrite(LED1, LOW);
  digitalWrite(LED2, LOW);
  digitalWrite(LED3, LOW);
  digitalWrite(LED4, LOW);
}

void beep(int onTime, int offTime) {
  digitalWrite(BUZZER, HIGH);
  delay(onTime);
  digitalWrite(BUZZER, LOW);
  delay(offTime);
}

long getDistance() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long dur = pulseIn(ECHO, HIGH);
  return dur / 58;
}

String getZone(long cm) {
  if      (cm > 50)  return "SAFE";
  else if (cm > 30)  return "NEAR";
  else if (cm > 15)  return "CAUTION";
  else if (cm > 5)   return "WARNING";
  else               return "STOP";
}

void loop() {
```

```
  long   cm   = getDistance();
  String zone = getZone(cm);

  allLedsOff();
  digitalWrite(BUZZER, LOW);

  // Print ONLY when distance changes > 2cm OR zone changes
  if (abs(cm - lastDistance) > 2 || zone != lastZone) {
    Serial.print("Distance: ");
    Serial.print(cm);
    Serial.print(" cm  |  ");
    if      (zone == "SAFE")    Serial.println("SAFE        [*---]  No beep");
    else if (zone == "NEAR")    Serial.println("NEAR        [**--]  Slow beep");
    else if (zone == "CAUTION") Serial.println("CAUTION     [***-]  Medium beep");
    else if (zone == "WARNING") Serial.println("WARNING!    [****]  Fast beep");
    else                        Serial.println(">>> STOP! STOP! STOP! <<<");
    lastDistance = cm;
    lastZone     = zone;
  }

  // LEDs and buzzer always active
  if (zone == "SAFE") {
    digitalWrite(LED1, HIGH);

  } else if (zone == "NEAR") {
    digitalWrite(LED1, HIGH);
    digitalWrite(LED2, HIGH);
    beep(100, 400);

  } else if (zone == "CAUTION") {
    digitalWrite(LED1, HIGH);
    digitalWrite(LED2, HIGH);
    digitalWrite(LED3, HIGH);
    beep(200, 250);

  } else if (zone == "WARNING") {
    digitalWrite(LED1, HIGH);
    digitalWrite(LED2, HIGH);
    digitalWrite(LED3, HIGH);
    digitalWrite(LED4, HIGH);
    beep(300, 100);

  } else {                          // STOP
    digitalWrite(LED4, HIGH);
    digitalWrite(BUZZER, HIGH);
    delay(200);
    digitalWrite(LED4, LOW);
    delay(200);
    digitalWrite(LED4, HIGH);
    delay(200);
  }
  delay(100);
}
```

6. How the Code Works

Step 1 — Trigger pulse

The Arduino pulls the TRIG pin HIGH for 10 microseconds. This causes the HC-SR04 to emit 8 ultrasonic bursts at 40 kHz frequency.

Step 2 — Echo measurement

The sound waves travel out, hit the object, and bounce back. The ECHO pin stays HIGH for exactly as long as the round trip takes. pulseIn(ECHO, HIGH) measures this duration in microseconds.

Step 3 — Distance calculation

Distance in cm = duration / 58. This comes from the speed of sound (340 m/s) divided by 2 (pulse travels to object AND back), converted to centimetres per microsecond.

Step 4 — Zone decision

The getZone() function checks the cm value against thresholds and returns a zone name string (SAFE / NEAR / CAUTION / WARNING / STOP).

Step 5 — Smart serial printing

The code compares the new reading with lastDistance and lastZone. Serial output is printed ONLY if the distance changed by more than 2 cm or the zone changed. This prevents the terminal from flooding with repeated identical lines.

Step 6 — LEDs and buzzer

LEDs are always reset with allLedsOff() first, then only the correct LEDs for that zone are turned on. The beep() function controls the buzzer with different ON/OFF timing per zone, making the beeps faster as the object gets closer — exactly like a real parking sensor.

Key design feature: The beep() function takes two arguments — onTime and offTime. By varying these, each zone produces a distinctly different rhythm: slow and calm for NEAR, fast and urgent for WARNING, and a solid continuous alarm for STOP.

7. Simulation Screenshots

The following screenshots were taken during the Proteus 8 simulation. Each image shows a different distance zone with the corresponding LEDs lit and the Virtual Terminal output.

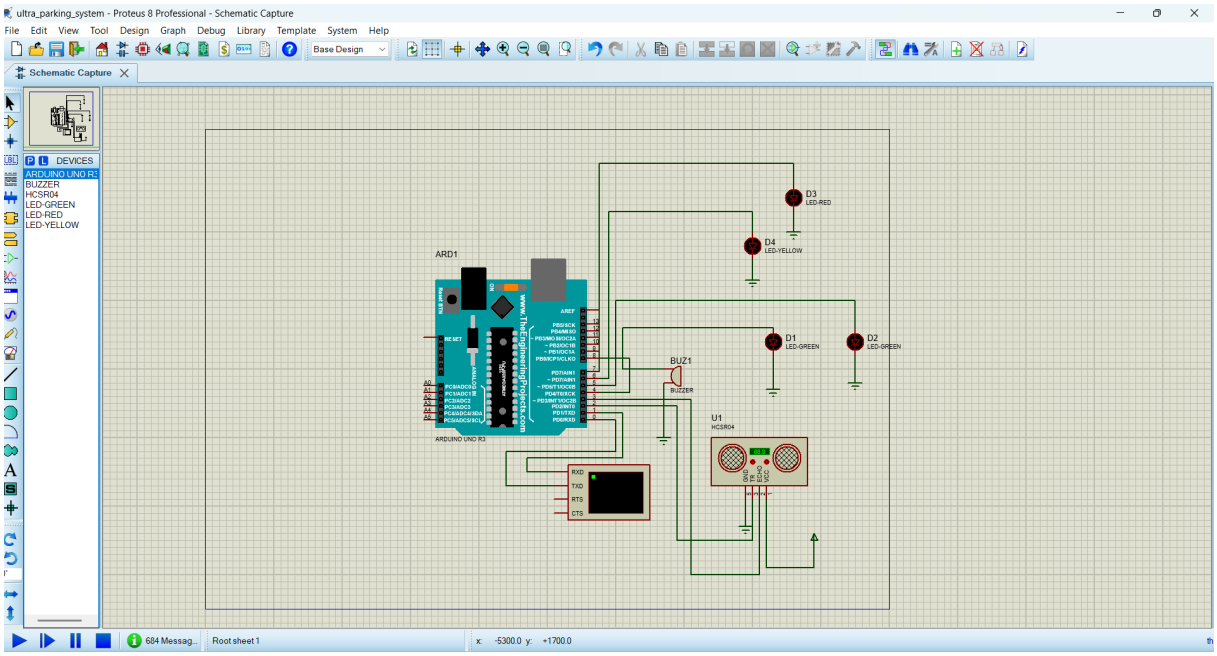

Figure 1 — Proteus schematic (no simulation running)

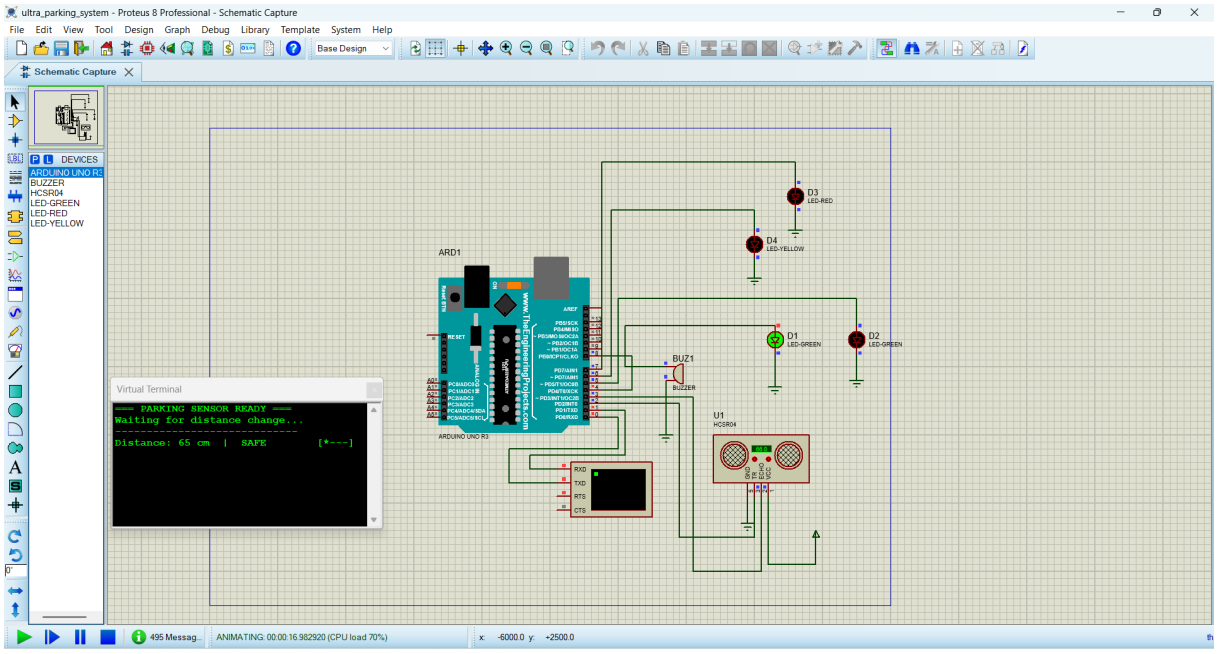

Figure 2 — Simulation started, first reading: 65 cm SAFE

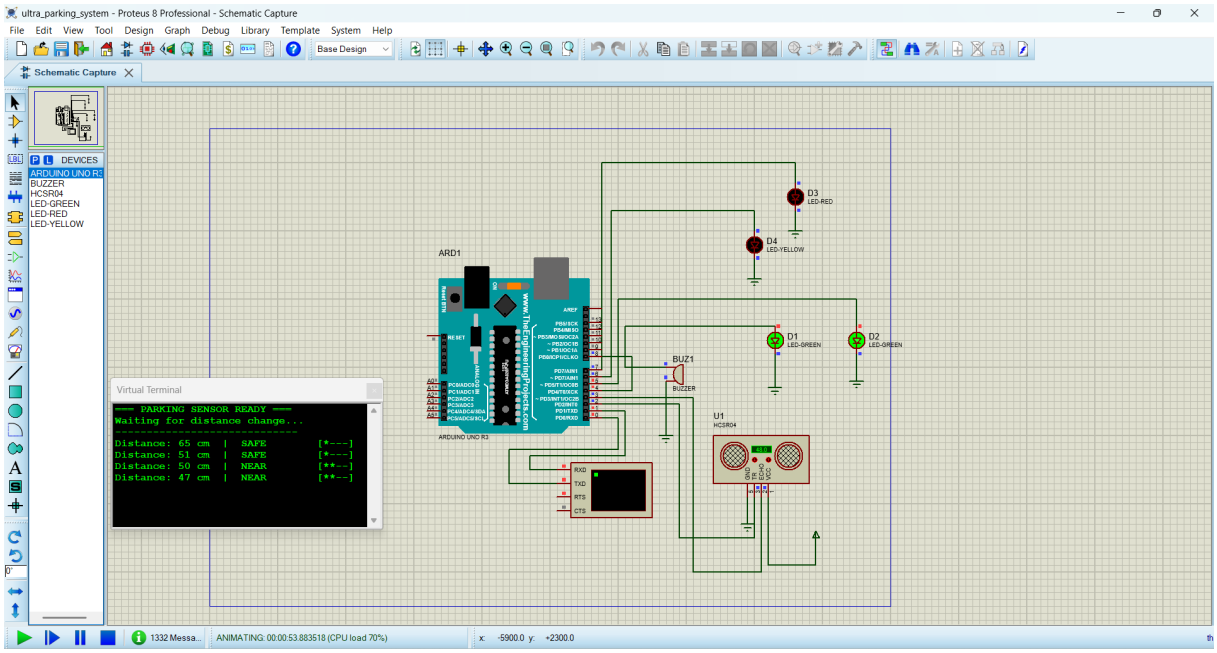

Figure 3 — Distance decreasing: SAFE to NEAR zone

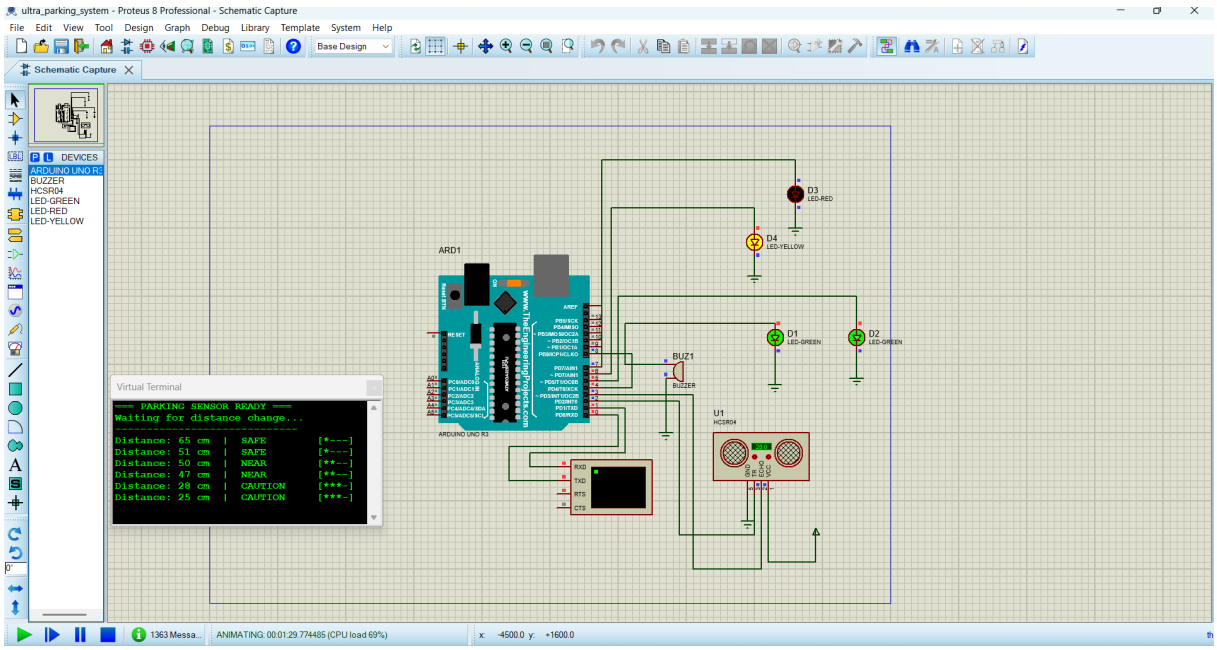

Figure 4 — Full zone progression: SAFE → NEAR → CAUTION

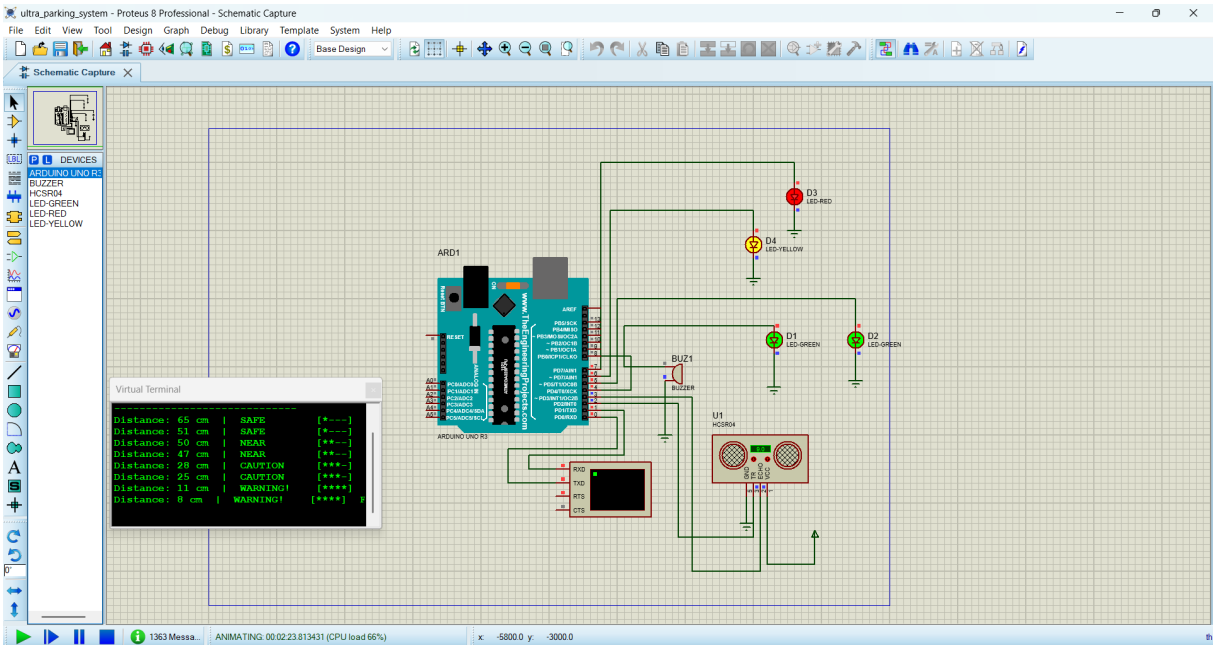

Figure 5 — WARNING and CAUTION zones active, LEDs lit

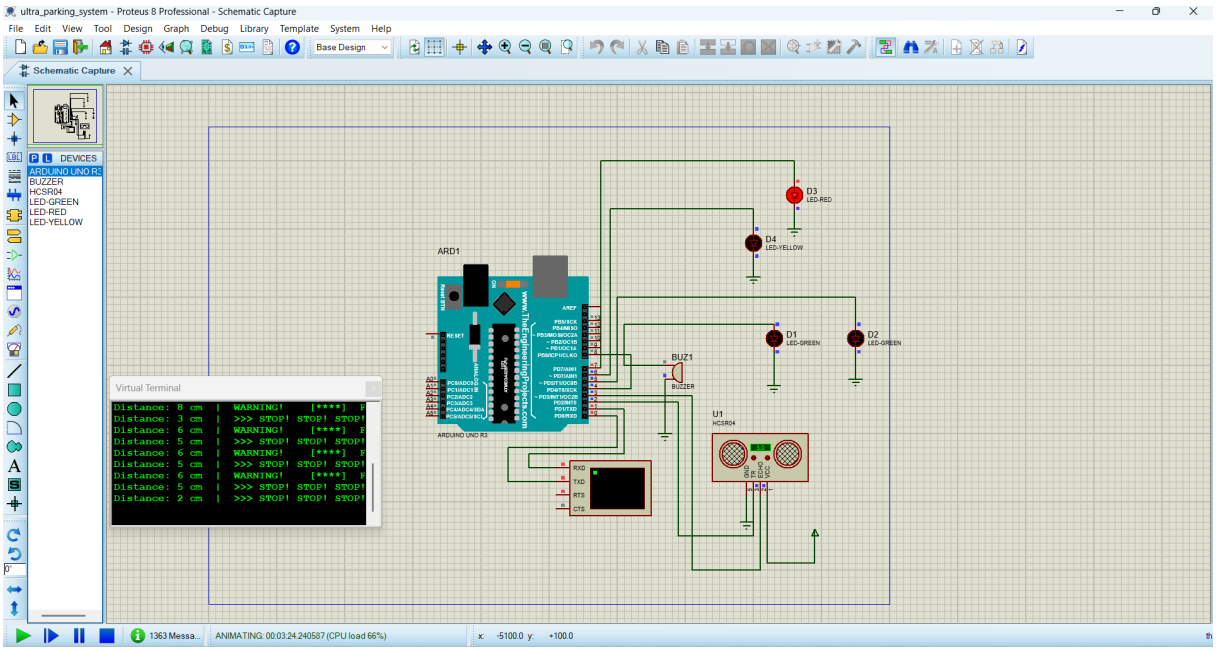

Figure 6 — STOP zone triggered, solid buzzer, RED LED flashing

8. Virtual Terminal Output Explained

The Virtual Terminal in Proteus acts as the serial monitor. It is connected to the Arduino TX (D1) pin with RXD, set to 9600 baud, 8 data bits, no parity, 1 stop bit. Below is the typical output produced as an object moves from far to close:

```
=== PARKING SENSOR READY ===
Waiting for distance change...
----------------------------
Distance: 65 cm  |  SAFE        [*---]  No beep
Distance: 51 cm  |  SAFE        [*---]  No beep
Distance: 50 cm  |  NEAR        [**--]  Slow beep
Distance: 47 cm  |  NEAR        [**--]  Slow beep
Distance: 28 cm  |  CAUTION     [***-]  Medium beep
Distance: 25 cm  |  CAUTION     [***-]  Medium beep
Distance: 11 cm  |  WARNING!    [****]  Fast beep
Distance:  8 cm  |  WARNING!    [****]  Fast beep
Distance:  3 cm  |  >>> STOP! STOP! STOP! <<<
```

The [*---] bar indicator fills up progressively as the object approaches, giving a quick visual summary of the current zone even without reading the zone name text.

9. Conclusion

This simulation successfully demonstrates a fully functional ultrasonic parking sensor system in Proteus 8 Professional. The HC-SR04 sensor accurately measures distance, the Arduino processes the reading and maps it to five distinct zones, and the output is communicated through both physical indicators (LEDs and buzzer) and a serial terminal.

Key achievements of this project include: progressive LED bar indication, variable beep speed matching real parking sensor behaviour, smart serial printing that only updates on meaningful changes, and clean modular code structure using dedicated functions (getDistance, getZone, beep, allLedsOff) for maintainability.

The simulation can be directly transferred to real hardware by uploading the same .hex file to a physical Arduino Uno with the same wiring, since Proteus accurately models the timing and I/O behaviour of the ATmega328P microcontroller.